

Software Configuration Management Plan of GoldFolks Software		
Doc # 1	Version: 1.0	Page 1 / 10

NexusNTU

Software Configuration Management Plan

Version 1.0

24th October 2025

Prepared by:

*Shen Jia Cheng
Vu Thao Nguyen
Akshay Krishna
Aishwarya Anand
Nguyen Phuong Linh
Tran Son Viet*

Software Configuration Management Plan of GoldFolks Software		
Doc # 1	Version: 1.0	Page 2 / 10

TABLE OF CONTENTS

1	Identification	2
1.1	<i>Document overview</i>	2
1.2	<i>Abbreviations and Glossary</i>	2
1.2.1	Abbreviations	2
1.2.2	Glossary	2
1.3	<i>References</i>	2
1.3.1	Project References	2
1.3.2	Standard and regulatory References	2
1.4	<i>Conventions</i>	2
2	Organization	2
2.1	<i>Activities and responsibilities</i>	3
2.1.1	Decisions process and Accountabilities	3
3	Configuration identification	4
3.1	<i>Identification rules</i>	4
3.1.1	Identification rules of configuration items	4
3.1.2	Identification rules of SOUPs	4
3.1.3	Identification rules of documents	4
3.1.4	Identification rules of a media	5
3.2	<i>Reference configuration identification</i>	5
3.3	<i>Configuration Baseline Management</i>	5
4	Configuration control	5
4.1	<i>Change Management</i>	5
4.2	<i>Interface Management</i>	6
4.3	<i>Evolutions control of SOUP items</i>	6
5	Configuration support activities	6
5.1	<i>Configuration Status Accounting</i>	6
5.1.1	Evolutions traceability	6
5.1.2	Setting up Configuration status	6
5.1.3	Configuration status diffusion	6
5.1.4	Configuration status records storage	6
5.2	<i>Configuration audits</i>	7
5.3	<i>Reviews</i>	7
5.4	<i>Configuration management plan maintenance</i>	7

Software Configuration Management Plan of GoldFolks Software		
Doc # 1	Version: 1.0	Page 3 / 10

1 Identification

This document is the Software Configuration Management (SCM) Plan of the NexusNTU Project.

It defines the configuration management framework, processes, and responsibilities adopted by the Team NexusNTU to ensure consistency, integrity, and traceability across all project artifacts during the entire software development lifecycle.

1.1 Document Overview

The SCM Plan specifies the policies, procedures, and conventions used to perform configuration identification, control, status accounting, and auditing for the NexusNTU software system. It aims to ensure that all configuration items (CIs)—including code, documentation, test scripts, and deployment artifacts—are properly tracked and versioned.

1.2 Abbreviations and Glossary

1.2.1 Abbreviations

Abbreviation	Full Form
CM	Configuration Management
CMS	Configuration Management System
SCM	Software Configuration Manager
SCI	Software Configuration Item
SOUP	Software of Unknown Pedigree/Provenance
VDD	Version Description Document
QA	Quality Assurance

1.2.2 Glossary

Word/Phrase	Explanation
Version	A particular form of document/application that differs in certain aspects from an earlier form or other forms of the same thing
Branch	To develop a document/application independently in parallel, with the possibility of merging in the future.
Merge	To combine different branches of work into a main branch

1.3 References

1.3.1 Project References

#	Document Identifier	Document Title
1	SRS	System Requirement Specifications
2	QP	Quality Plan
3	RMP	Risk Management Plan
4	PP	Project Plan

Software Configuration Management Plan of GoldFolks Software		
Doc # 1	Version: 1.0	Page 4 / 10

1.3.2 Standard and Regulatory References

#	Document Identifier	Document Title
1	SCM1	IEEE 828-2012 - IEEE Standard for Configuration Management in Systems and Software Engineering
2	SSCM1	IEEE Standard for Software Configuration Management Plans

1.4 Conventions

Typeface	Usage	Example
Bold	To assert emphasis on the text. Used in headers and titles as well.	1.4 Conventions
<u>Underline</u>	To help draw attention to the text. Also used to represent hyperlink.	<u>Refer to NexusNTU SCM repository.</u>
Highlighted	To assert special emphasis on important information in a text.	Total Cost: \$1500.50
Red	To indicate comments and mark out text that requires changes.	This document ... (Maybe change this part to this instead)
<i>Italics</i>	To assert minor emphasis on the text. Also used to refer to file names.	For more information, you can refer to our <i>Configuration Management Plan</i> instead.

2 Organization

The responsibilities of software configuration management are shared by members of the project in the following structure:

Project Manager: Aishwarya Anand
Release Engineer/Manager: Nguyen Phuong Linh
Front-end Developer: Akshay Krishna
Back-end Developer: Tran Son Viet
QA Manager: Shen Jia Cheng
QA Engineer: Vu Thao Nguyen

2.1 Activities and Responsibilities

The functions required to manage the configuration of the software and responsibilities are listed below.

Activities when setting up the project	Person responsible
Identify the configuration items	SCM
Install the bug repository tool and set up the database	SCM
Install the software configuration repository tool and set up the database	SCM
Manage and structure the reference space	SCM
Define the configuration processes	SCM

Software Configuration Management Plan of GoldFolks Software

Doc # 1

Version: 1.0

Page 5 / 10

Activities during the project lifecycle	Person responsible
Export components for modification, test or delivery	SCM
Set under control validated components	SCM
Create version, write version delivery document	SCM
Approve reference configurations	Project Manager
Verify version to be delivered and authorize deliveries	Project Manager
Backup spaces	SCM
Do configuration audits	QA Manager
Inspect configuration records	QA Manager
Archive reference version	SCM

Management activities	Person responsible
Manage versions and archives	SCM
Manage configuration records	SCM
Produce reports and statistics	SCM
Manage reference space and its access control list	SCM
Manage spaces backup and archive media	SCM
Manage quality reports	QA Manager

2.1.1 Decisions' Process and Responsibilities

Responsibilities during reviews, audits and approvals are listed below:

At the end of an activity of the project:

Activities	Person Responsible
Do a configuration freeze	SCM
Present a configuration state of the components impacted by the activity	SCM
Present a documentation state of the components impacted by the activity	SCM

During a configuration management process audit:

Activities	Person Responsible
Do the configuration management process audit	Project Manager
Present the records of the configuration management process	SCM
Present the quality records of the configuration management process	QA Manager
Present the records of the documentation management process	SCM

Software Configuration Management Plan of GoldFolks Software		
Doc # 1	Version: 1.0	Page 6 / 10

3 Configuration Identification

3.1 Identification Rules

3.1.1 Identification Rules of Configuration Items

3.1.1.1

3.1.1.2 Identification of a Configuration Item

The format for identification of a configuration item is:

Nexus_Vm.n.p where

- “_V” is the delimiter for version number
- “m” = MAJOR (big, incompatible change)
- “n” = MINOR (new features, still compatible)
- “p” = PATCH (bug/security fixes, no new features)

3.1.1.3 Version Number of a Configuration Item

Rule (before any delivery): assign or bump the version.

- Bump MAJOR ($m \rightarrow m+1$, reset n,p to 0) when we break backward compatibility (e.g., API or DB breaking change).
- Bump MINOR ($n \rightarrow n+1$, reset p to 0) when we add features that stay compatible.
- Bump PATCH ($p \rightarrow p+1$) when we fix bugs or security issues only.
- Pre-release identifiers (-alpha, -beta, -rc) are used for test builds before final.

3.1.2 Identification Rules of SOUPs

3.1.2.1 Identification of a SOUP

Prefer the vendor’s own ID (so audits match upstream):

- Node/Frontend (npm): name@version (e.g., react@18.3.1, express@4.19.2)
- Mapbox: mapbox-gl@X.Y.Z plus Style ID in Mapbox Studio.
- Dialogflow / Gemini: Dialogflow Agent version and model name (e.g., gemini-1.5-pro).
- MongoDB: server version and Node driver version.

3.1.2.2 Version number of a SOUP

- Use the vendor version exactly (pin; no latest, no caret ^).

3.1.3 Identification Rules of a documents

3.1.3.1 Description of documents identifiers

Scheme:

NEXUS_<document type>_<document number>_<revision index>

- <document type> (set aligned to your proposal & deliverables):
 - PROPOSAL – project proposal
 - SRS – Software Requirements Specification
 - SDD – Software Design Description (incl. MVC & data model)
 - API – API specification (REST endpoints, contracts)
 - QAP – Quality Assurance Plan
 - STP – Software Test Plan
 - STR – Software Test Report

Software Configuration Management Plan of GoldFolks Software		
Doc # 1	Version: 1.0	Page 7 / 10

- PLAN_SCM – Configuration Management Plan
- PLAN_CHANGE – Change Management Plan
- PLAN_RELEASE – Release Plan OPS
- <document number>: incremental 2-digit per type (01, 02, 03, ...).
- <revision index>: approved iteration V1, V2,

3.1.3.2 Definition and evolution of the revision index

Rule (must do before any delivery): every document file needs a revision index (like V1, V2). If we change the document, we increment the index before sharing it again.

How we bump the revision index:

- Major content change (new sections, changed scope, updated requirements or design): new revision → Vx → Vx+1.
- Minor change (typos, formatting, link fix that doesn't change meaning): we still bump to keep audit trails → Vx → Vx+1.
- Drafts (not approved yet): optional draft marker V0.y (e.g., V0.3). When approved, move to V1.
- Cross-doc consistency: if a document update forces another doc to change (e.g., API changes require Test Plan update), both get new revisions.

3.1.4 Identification rules of a media

3.1.4.1 Internal identification

Pattern:

<configuration item identification>/<media>/<volume>

- <configuration item identification>: the tagged version, e.g., nexus-platform-v1.3.0 or nexus-api-v1.3.0.
- <media>: media identifier
 - PKG for a signed package file (ZIP/TAR)
 - IMG for a Docker image archive
 - CLOUD for an uploaded artifact set (release in Git/GitHub/GitLab)
- <volume>: 001, 002, ... if delivery spans multiple media, archives.

3.2 Reference Configuration Identification

Each reference configuration is defined by:

- An identifier,
- Its content listed in the corresponding Version Delivery Description document,
- The acceptance or validation reviews associated with the building of the reference configuration.

A reference configuration is established for each design review and each test review of the project.

3.3 Configuration Baseline Management

What baselines we use (and when):

- Functional Baseline (FBL) — what the system must do
 - Content: approved SRS, user stories (frozen for the release), high-level use cases, non-functional requirements (performance, security), and agreed API surface (endpoints list, not detailed payloads).
 - When: after Requirements, Scope review (e.g., end of Inception / Sprint 0).

Software Configuration Management Plan of GoldFolks Software		
Doc # 1	Version: 1.0	Page 8 / 10

- Control: any change needs a change request (CR) + product owner approval; update SRS and bump revisions.
- Allocated Baseline (ABL) — how the system will do it
 - Content: approved SDD/ARCH, detailed API spec (payload contracts), DB schema & SCHEMAS migration set, external interfaces (Mapbox, NTU services), security design, and test strategy mapping to features.
 - When: after Architecture & Interface review (early construction phase).
 - Control: changes require tech lead + PO approval; update SDD/API/SCHEMAS and bump revisions; re-run impact analysis (tests, data migrations).
- Product Baseline (PBL) — the built thing we ship
 - Content: completed and accepted components (WEB, API, ADAPTERS, CHATBOT, MAP, SCHEMAS), deployment/OPS docs, VDD, STR (test report) with pass/fail evidence, SBOM, and licenses.
 - When: after System/Acceptance Test passes for the release candidate; confirmed in Release Review.
 - Control: PBL is immutable; if fixes are needed, create a new product baseline (e.g., nexus-platform-v1.3.1) with its own VDD & STR.

4 Configuration Control

Configuration control is used for managing configuration changes of our software application and related documents.

4.1 Change Management

It is inevitable for changes to occur in software projects due to changing target users' demands or bugs and errors. The process for controlling the changes to the baselines and for tracking the implementation of those changes are listed as follows:

Problem resolution:

- When a change request is accepted by the project manager, a branch is created in the CMS
- The branch identification is done as per the configuration item identification protocol
- Branch content is where the required changes will go to, before going through the rest of the change management workflow

Multiple configuration:

- When a change request is accepted by the project manager, a branch is created in the CMS
- The branch identification is also done as per the configuration item identification protocol
- Branch content is where the required changes will go to, before going through the rest of the change management workflow

4.2 Interface Management

Nexus will interact with the following 3rd parties:

- MongoDB
- Gemini
- Google Map API

Thus, it will require respective interfaces for each 3rd party.

Following is the procedure to identify the interface requirements for each interface:

- Identify the interface
- Identify the characteristics of the interface

Software Configuration Management Plan of GoldFolks Software		
Doc # 1	Version: 1.0	Page 9 / 10

- Define the interface responsibilities

4.3 Evolutions control of SOUP items

- Pin exact versions; commit lockfiles; keep an SBOM with name, version, license, source, usage.
- Do routine updates in a monthly window via a dedicated branch and PR with test results.
- Fast-track security updates anytime.
- For each update: run CI (tests, lint, build). Use a canary for risky changes; monitor; roll back to the last tag if needed.
- Track CVEs: open a CR, assess, patch or mitigate, test, hotfix tag, and note in the VDD and security log.
- Freeze SOUPs during critical phases; allow only urgent security fixes.

5 Configuration Support Activities

Following the same structure and goal as the reference plan, this section outlines how we record, report, audit, and maintain configuration for the NexusNTU web project. We also customize roles, tools, and artifacts to fit our stack (GitHub, Firebase, Maps/Places, FX, Links Registry).

5.1 Configuration Status Accounting (CSA)

At specified milestones (the conclusion of each sprint staging drop; before Lab 4/5 deliverables), Configuration Status Accounting (CSA) keeps track of the status of each configuration item (CI), including source code, infrastructure/config, test artifacts, data schemas, and controlled documents. Traceability, appropriate storage, and controlled version diffusion are all guaranteed by CSA.

5.1.1 Evolutions Traceability

- **Documents (Project Plan, SRS, Use Cases, Design):** Every modification is noted with a "modification sheet" entry that includes the change ID, author, and summary, as well as visible revision marks. A one-line delta for each document that changed is included in the weekly status.
- **Source files (web app, services, scripts):** Pull requests (PRs) are the authoritative change record that describes the modification with enough detail to identify the affected parts, while Git commits refer to task or issue IDs.
- **Configuration items (releases/environments):** A Version Description Document (VDD) that details the scope, issues resolved, database migrations, and links to the included PRs is included with every release.

5.1.2 Setting Up Configuration Status

We record the following information for every CI/release:

- tag/label,
- semantic version (such as v1.2.0),
- VDD creation date.

The VDD is the main control document that summarizes features and changes since the last release and is drafted by the Software Configuration Manager (SCM) (one VDD per unique release form).

Software Configuration Management Plan of GoldFolks Software		
Doc # 1	Version: 1.0	Page 10 / 10

5.1.3 Configuration Status Diffusion

After confirming test results and known problems, the Quality Manager co-signs the VDD, which is prepared by the SCM and distributed along with an executive one-page summary of major changes. Both have been added to the "Releases" section of the project wiki.

5.1.4 Configuration Status Records Storage

Records are kept in a versioned, structured archive that includes: (1) authorized change requests/issues (by ID), (2) controlled documents, (3) VDDs, and (4) snapshots of the configuration state taken over time. Locations of storage:

- **Repo:** /docs/releases/ (VDDs, manifests)
- **Wiki:** Releases index (human-readable)
- **Cloud folder:** "Configuration/Records" immutable export per release

5.2 Configuration Audits

To make sure that this plan and the baselined artifacts are being followed, we perform audits:

- **Functional Configuration Audit (FCA):** Independent evaluation that the product functions match the approved requirements/specifications (such as use-case acceptance criteria for Hub/Links/Maps/FX/Chatbot).
- **Physical Configuration Audit (PCA):** Verification that as-built items (code, configs, data schemas) map exactly to their specifications.
- **In-Process CM Audits:** Ongoing checks of compliance with CM policies (branching, reviews, tagging, VDD completeness). A configuration audit is required before any classroom release/demo. Each audit has an Audit Plan stating objective, items under audit, schedule, procedures, participants (by role), required documentation, deficiency recording, corrective-action tracking, and approval criteria.

5.3 Reviews

Requirement conformance, consistency across document levels, and traceability across baselines are all ensured by technical reviews (design reviews, sprint reviews, and release readiness). Before approving proposed changes and baselines, the SCM confirms that they comply with identification/control rules; the Quality Manager confirms the evidence.

5.4 Configuration Management Plan Maintenance

- This CM Plan is monitored by the SCM; it is reviewed and updated every month or whenever there is a major change in the process or tools (such as a new CI step).
- Project Manager approval is required for changes, after which the SCM notifies the team via email and wiki update, and publishes the updated plan.
- Maintenance entails modifying schedules, resources, activities (identification, control, auditing, and reporting), and organization. Every update complies with SC3040's SCM plan guidelines.